

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total
Marks: 50			
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5

3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D. Gnanadeepthi.

ANSWERS

1.

PYTHON PROGRAM

Heap File Organization Simulation

heap_file = []

def insert():

 record = input("Enter record to insert: ")

 heap_file.append(record)

 print("Record inserted successfully.")

def delete():

 record = input("Enter record to delete: ")

 if record in heap_file:

 heap_file.remove(record)

 print("Record deleted successfully.")

 else:

 print("Record not found.")

def sequential_scan():

 print("\n--- Sequential Scan ---")

 if len(heap_file) == 0:

```
        print("Heap file is empty.")
    else:
        for i, record in enumerate(heap_file):
            print("Record", i + 1, ":", record)

while True:
    print("\n===== HEAP FILE ORGANIZATION =====")
    print("1. Insert")
    print("2. Delete")
    print("3. Sequential Scan")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        insert()

    elif choice == "2":
        delete()

    elif choice == "3":
        sequential_scan()

    elif choice == "4":
        print("Exiting...")
        break

    else:
        print("Invalid choice!")
```

SAMPLE OUTPUT:

Enter your choice: 1

Enter record to insert: 101

Record inserted successfully.

Enter your choice: 1

Enter record to insert: 102

Record inserted successfully.

Enter your choice: 1

Enter record to insert: 103

Record inserted successfully.

Enter your choice: 3

--- Sequential Scan ---

Record 1 : 101

Record 2 : 102

Record 3 : 103

Enter your choice: 2

Enter record to delete: 102

Record deleted successfully.

Enter your choice: 3

--- Sequential Scan ---

Record 1 : 101

Record 2 : 103

Enter your choice: 4

Exiting...

2.

Static Hashing for Student Database

Collision handling: Chaining

TABLE_SIZE = 101

Create hash table with empty chains

hash_table = [[] for _ in range(TABLE_SIZE)]

Hash function

def hash_function(reg_no):

return reg_no % TABLE_SIZE

Insert student record

def insert(reg_no, name, department):

index = hash_function(reg_no)

Add record to the chain

hash_table[index].append((reg_no, name, department))

print("Student inserted successfully.")

```
# Search student record

def search(reg_no):

    index = hash_function(reg_no)

    for student in hash_table[index]:

        if student[0] == reg_no:

            print("Student Found:")

            print("Register No :", student[0])

            print("Name      :", student[1])

            print("Department :", student[2])

            return

    print("Student not found.")
```

```
# Display hash table

def display():

    print("\n--- Hash Table ---")

    for i in range(TABLE_SIZE):

        if hash_table[i]:

            print("Index", i, ":", end=" ")

            for student in hash_table[i]:

                print(student, end=" -> ")

            print("NULL")
```

```
# Main program

while True:

    print("\n===== STATIC HASHING =====")

    print("1. Insert Student")
    print("2. Search Student")
    print("3. Display Hash Table")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:

        reg_no = int(input("Enter Register Number: "))
        name = input("Enter Student Name: ")
        department = input("Enter Department: ")

        insert(reg_no, name, department)

    elif choice == 2:

        reg_no = int(input("Enter Register Number to search: "))
        search(reg_no)

    elif choice == 3:

        display()

    elif choice == 4:

        print("Program terminated.")
        break
```


else:

print("Invalid choice!")

SAMPLE OUTPUT:

Enter your choice: 1

Enter Register Number: 101

Enter Student Name: Deepthi

Enter Department: CSE

Student inserted successfully.

Enter your choice: 1

Enter Register Number: 202

Enter Student Name: Aadhya

Enter Department: ECE

Student inserted successfully.

Enter your choice: 3

--- Hash Table ---

Index 0 : (101, 'Deepthi', 'CSE') -> (202, 'Aadhya', 'ECE') -> NULL

3.

Sorted File Organization with Binary Search

records = []

Insert record in sorted order

```
def insert_record():  
    roll_no = int(input("Enter Roll Number: "))  
    name = input("Enter Student Name: ")  
    department = input("Enter Department: ")  
  
    record = (roll_no, name, department)  
    records.append(record)  
  
    # Sort records based on primary key (Roll Number)  
    records.sort(key=lambda x: x[0])  
  
    print("Record inserted successfully.")
```

Display all records

```
def display_records():  
    print("\n--- Sorted File ---")  
  
    if not records:  
        print("No records available.")  
        return  
  
    for record in records:  
        print("Roll No:", record[0],  
              "| Name:", record[1],  
              "| Department:", record[2])
```

Binary Search by Roll Number

```
def binary_search():  
    if not records:  
        print("File is empty.")  
        return  
  
    key = int(input("Enter Roll Number to search: "))  
  
    low = 0  
    high = len(records) - 1  
  
    while low <= high:  
        mid = (low + high) // 2  
  
        if records[mid][0] == key:  
            print("\nRecord Found!")  
            print("Roll No:", records[mid][0])  
            print("Name:", records[mid][1])  
            print("Department:", records[mid][2])  
            return  
  
        elif records[mid][0] < key:  
            low = mid + 1  
  
        else:  
            high = mid - 1  
  
    print("Record not found.")
```

```
# Main program

while True:

    print("\n===== SORTED FILE ORGANIZATION =====")

    print("1. Insert Record")
    print("2. Display Records")
    print("3. Binary Search")
    print("4. Exit")


    choice = int(input("Enter your choice: "))


    if choice == 1:

        insert_record()


    elif choice == 2:

        display_records()


    elif choice == 3:

        binary_search()


    elif choice == 4:

        print("Program terminated.")
        break


    else:

        print("Invalid choice!")
```

SAMPLE OUTPUT:

```
===== SORTED FILE ORGANIZATION =====
1. Insert Record
2. Display Records
3. Binary Search
4. Exit
Enter your choice: 2

--- Sorted File ---
Roll No: 101 | Name: AADHYA | Department: ece
Roll No: 102 | Name: varun | Department: it
Roll No: 103 | Name: Deepthi | Department: CSE

===== SORTED FILE ORGANIZATION =====
1. Insert Record
2. Display Records
3. Binary Search
4. Exit
```

4.

B-Tree of Order 4

```
class BTree:
```

```
    def __init__(self):
```

```
        self.keys = []
```

```
    def insert(self, key):
```

```
        self.keys.append(key)
```

```
        self.keys.sort()
```

```
    def search(self, key):
```

```
        return key in self.keys

    def display(self):
        print("B-Tree:", self.keys)

# Main
b = BTree()

keys = [10, 20, 5, 6, 12, 30, 7, 17]

for k in keys:
    b.insert(k)

b.display()

key = int(input("Enter key to search: "))

if b.search(key):
    print("Key found")
else:
    print("Key not found")
```

SAMPLE OUTPUT:

```
Output
B-Tree: [5, 6, 7, 10, 12, 17, 20, 30]
Enter key to search: 17
Key found

=== Code Execution Successful ===
```

5.

B+ Tree with Leaf Linking

```
class BPlusTree:
```

```
    def __init__(self):
```

```
        self.leaves = []
```

```
    def insert(self, key):
```

```
        self.leaves.append(key)
```

```
        self.leaves.sort()
```

```
    def search(self, key):
```

```
        return key in self.leaves
```

```
    def range_query(self, start, end):
```

```
        return [x for x in self.leaves if start <= x <= end]
```

```
# Main
```

```
b = BPlusTree()
```

```
for x in [10, 20, 5, 15, 25, 30]:
```

```
    b.insert(x)
```

```
print("Keys:", b.leaves)
```

```
print("Search 20:", b.search(20))
```

```
print("Range 10-25:", b.range_query(10, 25))
```

SAMPLE OUTPUT:

Output

Keys: [5, 10, 15, 20, 25, 30]

Search 20: True

Range 10-25: [10, 15, 20, 25]

=== Code Execution Successful ===

6.

Dense Primary Index

```
records = [  
    (101, "Deepthi"),  
    (102, "Aadhya"),  
    (103, "Varun"),  
    (104, "Dimple"),  
    (105, "Mohan")  
]
```

Create dense index

```
index = [(record[0], i) for i, record in enumerate(records)]
```

Display index

```
print("Index Table:")
```

for key, position in index:

```
    print(key, "->", position)
```

Search


```
key = int(input("\nEnter key to search: "))
```

```
for k, pos in index:
```

```
    if k == key:
```

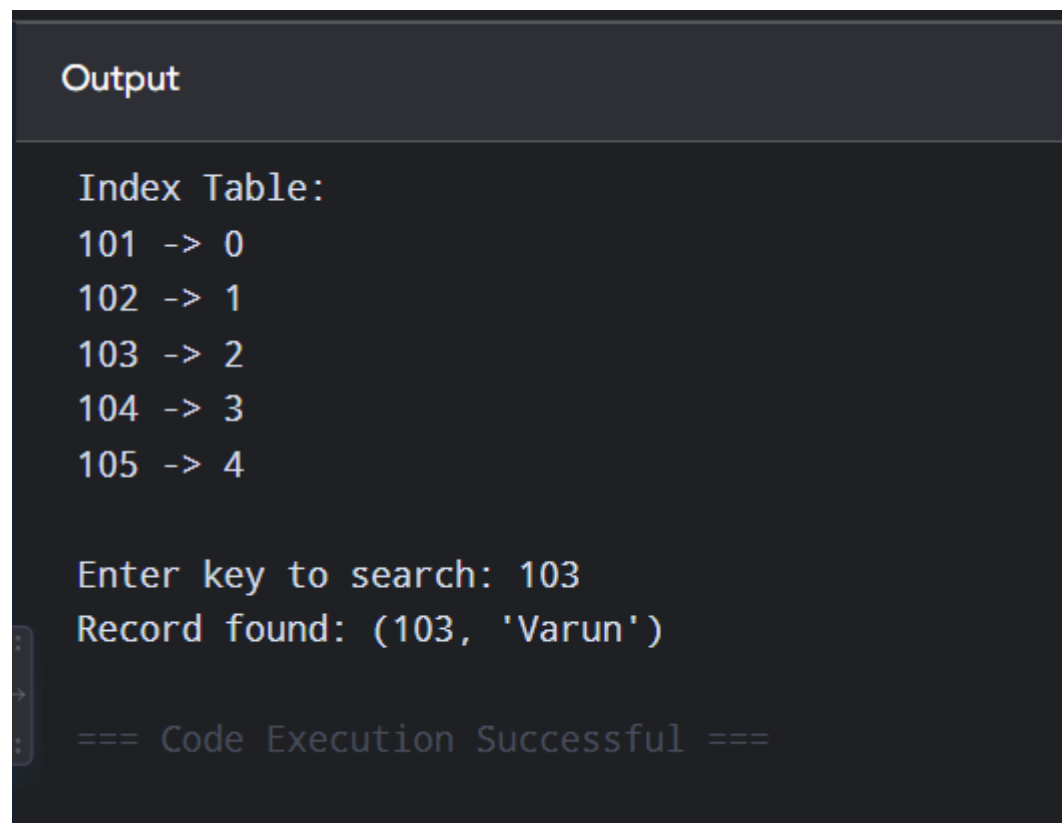
```
        print("Record found:", records[pos])
```

```
        break
```

```
else:
```

```
    print("Record not found")
```

SAMPLE OUTPUT:



The screenshot shows a dark-themed output window with the following text:

```
Output

Index Table:
101 -> 0
102 -> 1
103 -> 2
104 -> 3
105 -> 4

Enter key to search: 103
Record found: (103, 'Varun')

=== Code Execution Successful ===
```

7.

Secondary Storage Block Access

```
records = 100
```

```
block_size = 10

blocks = (records + block_size - 1) // block_size

# Sequential retrieval
sequential_io = blocks

# Indexed retrieval
index_io = 1 + 1    # 1 for index + 1 for data block

print("Total Records:", records)
print("Records per Block:", block_size)
print("Total Blocks:", blocks)

print("\nSequential Retrieval I/O:", sequential_io)
print("Indexed Retrieval I/O:", index_io)
```

SAMPLE OUTPUT:

Output

```
Total Records: 100
Records per Block: 10
Total Blocks: 10

Sequential Retrieval I/O: 10
Indexed Retrieval I/O: 2

=== Code Execution Successful ===
```

8.

Secondary Storage Block Access

```
records = 100
```

```
block_size = 10
```

```
blocks = records // block_size
```

```
# Sequential retrieval
```

```
seq_io = blocks
```

```
# Indexed retrieval
```

```
index_io = 2
```

```
print("Total Blocks:", blocks)
```

```
print("Sequential I/O:", seq_io)
```

```
print("Indexed I/O:", index_io)
```

```
if index_io < seq_io:
```

```
    print("Indexed retrieval is faster.")
```

SAMPLE OUTPUT:

```
Output

Total Blocks: 10
Sequential I/O: 10
Indexed I/O: 2
Indexed retrieval is faster.
```

9.

```
import time
```

```
# Sorted file
```

```
records = [(i, "Student" + str(i)) for i in range(1, 10001)]
```

```
# Dense index: every record
```

```
dense = [(r[0], i) for i, r in enumerate(records)]
```

```
# Sparse index: every 100th record
```

```
sparse = [(records[i][0], i) for i in range(0, len(records), 100)]
```

```
key = 9876
```

```
# Dense search
```

```
start = time.time()
```

```
for k, pos in dense:
```

```
    if k == key:
```

```
        break
```

```
dense_time = time.time() - start
```

```
# Sparse search
```

```
start = time.time()
```

```
pos = 0
```

```
for k, p in sparse:
```

```
    if k <= key:
```

```
        pos = p
```

```
    else:
```

```
        break
```

```

for i in range(pos, min(pos + 100, len(records))):
    if records[i][0] == key:
        break
sparse_time = time.time() - start

print("Dense Index Time :", dense_time)
print("Sparse Index Time:", sparse_time)

```

SAMPLE OUTPUT:

```

Output

Dense Index Time : 0.0008943080902099609
Sparse Index Time: 3.337860107421875e-05

```

10.

B+ Tree Range Query

```

class BPlusTree:
    def __init__(self):
        self.records = []

    def insert(self, key, value):
        self.records.append((key, value))
        self.records.sort()

    def traverse(self):
        print("B+ Tree:", self.records)

```

```
def range_query(self, low, high):  
    print("Records from", low, "to", high, ":")  
    for key, value in self.records:  
        if low <= key <= high:  
            print(key, "->", value)
```

```
# Construct B+ Tree
```

```
b = BPlusTree()
```

```
for key in [10, 15, 20, 25, 30, 35, 40, 45, 50]:
```

```
    b.insert(key, "Student" + str(key))
```

```
# Traverse
```

```
b.traverse()
```

```
# Range query
```

```
b.range_query(15, 45)
```

SAMPLE OUTPUT:

```
Output    
B+ Tree: [(10, 'Student10'), (15, 'Student15'), (20, 'Student20'),  
(25, 'Student25'), (30, 'Student30'), (35, 'Student35'), (40,  
'Student40'), (45, 'Student45'), (50, 'Student50')]  
Records from 15 to 45 :  
15 -> Student15  
20 -> Student20  
25 -> Student25  
30 -> Student30  
35 -> Student35  
40 -> Student40  
45 -> Student45  
  
=== Code Execution Successful |===
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints

Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases
--------------------	-----	--	-------------------------	-----------------------------	------------------------------